1 Related work

The group's starting point was Nicolás Rocchia's survey of how existing formats represent a check that was expected to run and did not (Sep/0000). He asked four questions of each format: can it say "this check did not run" as a state distinct from pass and fail; must the producer say why; is the reason a controlled vocabulary or free text; and is the not-run state meant to be shown to the consumer or treated as silence. He found that the third state is old, that the last question separates the formats, and that none of them states its answer. The paragraphs below extend his table to the formats the paper is most often compared with.

Test-result formats. TAP version 14 marks a test point with a `SKIP` directive and an optional explanation, and tells harnesses to list skipped points and not to count them as failures (`https://testanything.org/tap-version-14-specification.html`; also row 1 of Sep/0000). JUnit XML, as described by its widely used schema, gives a test case a `skipped` element with an optional message and keeps `error` ("an unanticipated problem") apart from `failure` (`https://github.com/windyroad/JUnit-Schema`). The W3C Evaluation and Report Language (EARL) 1.0 Schema, a Working Group Note of 2017, has five outcome values: passed, failed, cantTell, inapplicable and untested ("the test has not been carried out") (`https://www.w3.org/TR/EARL10-Schema/`). EARL's cantTell is the nearest prior form of the cannot-tell answer the readers written from the v0.1 text give. None of these formats binds a result to a pinned checker revision, or makes the producer show a check that fails on purpose, as v0.1's section 5.4 does.

Analysis results and attestations. SARIF 2.1.0 gives each result a `kind`, one of whose values, `notApplicable`, says the rule "was not evaluated, because it does not apply to the analysis target", and records per invocation whether the tool ran successfully (`executionSuccessful`) (`https://docs.oasis-open.org/sarif/sarif/v2.1.0/`). The in-toto test-result predicate (v0.1) states a run's `result` as `PASSED`, `WARNED` or `FAILED`, lists passed, warned and failed tests by name, and requires the `configuration` the tests ran under (`https://github.com/in-toto/attestation/blob/main/spec/predicates/test-result.md`). It has no state for a test that did not run. Neither defines a negative control.

Evaluation harnesses for language models. HELM evaluates language models across many scenarios and metrics and publishes the raw prompts and completions behind its numbers (Liang et al., `https://arxiv.org/abs/2211.09110`). The LM Evaluation Harness writes results with `-output_path` and per-sample responses with `-log_samples`, and is itself archived under a DOI (10.5281/zenodo.10256836; `https://github.com/EleutherAI/lm-evaluation-harness`). Inspect writes an evaluation log whose top-level status is `started`, `success` or `error` (`https://inspect.aisi.org.uk/eval-logs.html`). These harnesses record what was run and what came out at the level of an evaluation; v0.1 records it at the level of a single check, with the reason when the check gave no verdict, and gives a reader rows by which a report can be rejected.

Checks built to fail. The question section 5.4 asks of a roll-up is an old one in two other fields. Mutation analysis judges a test suite by whether it fails on deliberately altered programs (DeMillo, Lipton and Sayward, "Hints on test data selection", IEEE Computer 11(4), 1978; survey: Jia and Harman, IEEE Transactions on Software Engineering 37(5), 2011); R3's 192 mutants, of which 44 serve as controls, are of this kind. Epidemiology uses negative controls, an exposure or outcome that should show no effect, to detect bias that an all-positive result would hide (Lipsitch, Tchetgen Tchetgen and Cohen, Epidemiology 21(3), 2010). Both measure a suite or a study. v0.1 asks the same question of a single report, and requires the answer, including "nothing", to be written into the report a reader receives.